Mobile Device Application Development

Assignment 01 – Android UI Layout Implementation

Objective

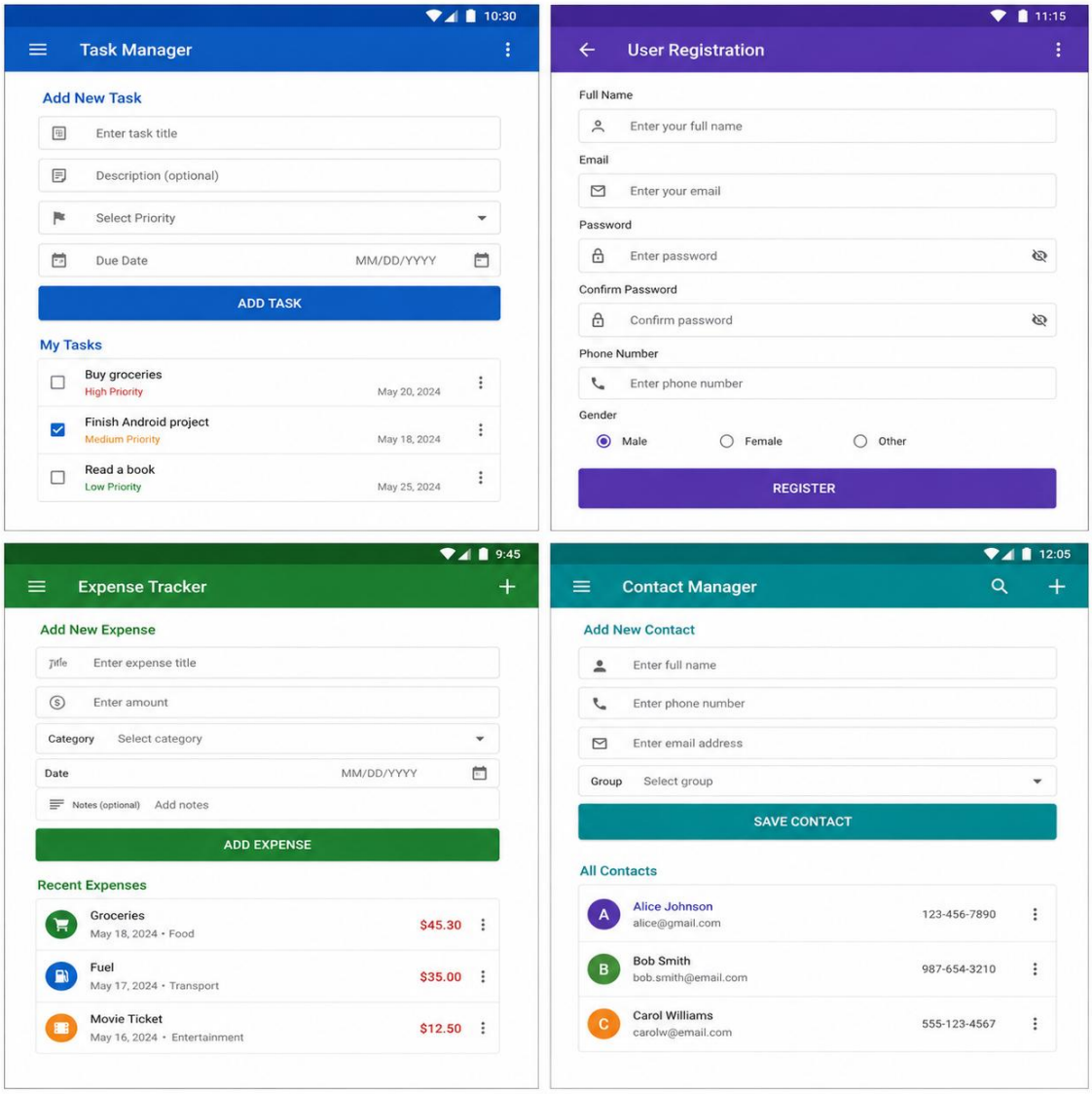
The objective of this assignment is to help students practice:

- Designing Android user interfaces using XML layouts
- Creating Android UI programmatically using Java code
- Understanding common Android widgets and layout structures
- Comparing declarative UI design (XML) and programmatic UI generation (Java)

Assignment Description

Students are required to implement **ONE Android application UI design** based on the assigned design number.

The four UI designs are shown in the provided image.



The designs are numbered as follows:

Position in Image	Design Number
Top-left	Design 1
Top-right	Design 2
Bottom-left	Design 3
Bottom-right	Design 4

Design Assignment Rule

Each student must determine their assigned design using the following rule:

Design Number = StudentID mod 4

Special case:

If StudentID mod 4 = 0 → Design 4

Examples:

Student ID	StudentID mod 4	Assigned Design
24120001	1	Design 1
24120002	2	Design 2
24120003	3	Design 3
24120004	0	Design 4

Requirements

Students must implement the assigned UI design in TWO ways.

Part 1 – XML Layout Implementation

Create the Android layout using XML.

Requirements

- Use standard Android widgets only
- Use proper layout structures
- Organize the interface clearly and logically
- The layout should visually resemble the assigned design

Example Widgets

- TextView
- EditText
- Button
- CheckBox
- RadioButton
- ImageView
- ListView / ScrollView
- LinearLayout / RelativeLayout / ConstraintLayout

Part 2 – Java-based UI Creation

Create the same interface programmatically using Java code.

Requirements

- Dynamically create all views/widgets in Java
- Set layout parameters programmatically
- Add widgets into parent layouts manually
- The final UI should resemble the XML version

Additional Requirement – Brief Report

Students must include a brief report (PDF or DOCX) describing their implementation process.

The report should include:

- A short explanation of how the assignment was implemented

- The layout/design strategy used
- Difficulties encountered during implementation
- How debugging/testing was performed
- Whether LLM/AI tools were used during development

If LLM/AI tools were used, students should briefly describe:

- Which tool(s) were used (e.g., ChatGPT, Gemini, Copilot, etc.)
- What kinds of assistance were requested
- How the generated suggestions/code were modified or adapted

Examples:

- asking for syntax explanations
- debugging help
- layout suggestions
- widget usage examples
- code improvement ideas

If no LLM/AI tools were used, students may simply state that the implementation was completed manually without AI assistance.

Purpose of This Requirement

The purpose of this report is:

- to encourage reflective learning,
- to help students practice responsible use of AI-assisted programming tools,
- and to better understand each student's development process.

The evaluation focuses on:

- understanding,
- implementation ability,
- and the ability to explain the submitted work,

rather than whether AI tools were used or not.

Submission Requirements

Students must submit:

1. XML layout file(s)
2. Java source file(s)
3. Brief report (PDF or DOCX)

Compress all files into ONE archive file:

StudentID.zip

or

StudentID.rar

Example:

24120001.zip

Recommended Submission Structure

```
StudentID.zip
│
├── XML/
│   └── activity_main.xml
│
├── JAVA/
│   └── MainActivity.java
│
├── REPORT/
│   └── report.pdf
│
└── README.txt (optional)
```

Important Notes

Academic Integrity & Use of LLM Tools

Students are encouraged to:

- Study Android documentation
- Read tutorials and references
- Use AI/LLM tools (such as ChatGPT, Gemini, Copilot, etc.) as learning assistants

However:

- Students must understand and modify the generated content
- Students should NOT simply copy generated code and submit it directly
- The submitted work should reflect the student's own understanding and implementation effort

Recommended Workflow

1. Understand the UI structure
2. Attempt implementation independently
3. Use LLM tools for:
    - syntax suggestions
    - debugging assistance
    - explanation of concepts
    - layout improvement ideas
4. Revise and complete the implementation yourself

During evaluation, students may be asked to explain:

- layout structure
- widget hierarchy
- Java code logic
- differences between XML and Java implementations

Therefore, students should ensure they fully understand their submitted work.

Evaluation Criteria

Criteria	Weight
XML layout correctness	35%
Java UI implementation	35%
Visual similarity to assigned design	20%
Code organization and readability	10%